TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

Semestre: Enero – Agosto 2026

CARRERA
Ingeniería Sistemas Computacionales

MATERIA
Patrones de diseño de software

TÍTULO
Composite

UNIDAD
3

ALUMNO(S) Y NO. DE CONTROL
Esmeralda Gamboa Hernandez
21210374
Velazquez Morales Lucero
-22210362
Vidales Hurtado Alitzel Dalhi
-21212371
Valdes Siqueiros Ivan Antonio
21210430

DOCENTE
MARIBEL GUERRERO LUIS

Tijuana Baja,California a 26 de Marzo del 2026

Introducción

El patrón Bridge se centra en desacoplar una abstracción de su implementación para que ambas puedan evolucionar de forma independiente, utilizando la composición en lugar de la herencia para evitar una explosión de clases. Por otro lado, el patrón Decorator permite añadir funcionalidades o responsabilidades adicionales a un objeto de manera dinámica y transparente, envolviéndolo en una serie de clases decoradoras sin alterar su estructura original. Mientras que el Bridge se utiliza principalmente en la fase de diseño para separar interfaces de plataformas o motores de ejecución, el Decorador es ideal para extender el comportamiento de objetos individuales en tiempo de ejecución sin afectar a otros objetos de la misma clase.

Código

```
import { useState } from 'react';
import { motion, AnimatePresence } from 'motion/react';
import { GraduationCap, Book, User, FileText, Sparkles } from 'lucide-react';
// ===== patron bridge =====
// Interfaz para materiales (implementación)
interface IMaterial {
obtenerDescripcion(): string;
}
// Implementación concreta del Bridge
class Libro implements IMaterial {
private nombre: string;
constructor(nombre: string) {
this.nombre = nombre;
}
obtenerDescripcion(): string {
return `Libro: ${this.nombre}`;
}
}
// ===== patron decorator =====
// Clase Materia que contiene los materiales
class Materia {
clave: string;
nombre: string;
maestro: string;
materiales: IMaterial[] = [];
constructor(clave: string, nombre: string, maestro: string) {
this.clave = clave;
this.nombre = nombre;
this.maestro = maestro;
}
}
// Componente abstracto base (Component del patrón Decorator)
```

```
abstract class TecnmComponent {
abstract get clave(): string;
abstract get descripcion(): string;
abstract obtenerMaterias(): Materia[];
}
// Componentes concretos (ConcreteComponent)
class IngSC extends TecnmComponent {
get clave(): string {
return 'ISC';
}
get descripcion(): string {
return 'Ingeniería en Sistemas Computacionales';
}
obtenerMaterias(): Materia[] {
return [];
}
}
class IngI extends TecnmComponent {
get clave(): string {
return 'INF';
}
get descripcion(): string {
return 'Ingeniería en Informática';
}
obtenerMaterias(): Materia[] {
return [];
}
}
class IngC extends TecnmComponent {
get clave(): string {
return 'ICB';
}
get descripcion(): string {
return 'Ingeniería en Ciberseguridad';
}
obtenerMaterias(): Materia[] {
return [];
}
}
// Decorator abstracto base (Decorator)
abstract class AgregadoDecorator extends TecnmComponent {
protected _tecnm: TecnmComponent;
constructor(t: TecnmComponent) {
super();
this._tecnm = t;
}
get clave(): string {
return this._tecnm.clave;
}
get descripcion(): string {
return this._tecnm.descripcion;
}
obtenerMaterias(): Materia[] {
return this._tecnm.obtenerMaterias();
}
}
```

```
// Decorators concretos (ConcreteDecorators)
class Fisica extends AgregadoDecorator {
obtenerMaterias(): Materia[] {
const lista = super.obtenerMaterias();
const materia = new Materia('FIS101', 'Física', 'Dr. López');
materia.materiales.push(new Libro('Física Serway'));
materia.materiales.push(new Libro('Física Universitaria - Sears y
Zemansky'));
lista.push(materia);
return lista;
}
}
class Calculo extends AgregadoDecorator {
obtenerMaterias(): Materia[] {
const lista = super.obtenerMaterias();
const materia = new Materia('MAT101', 'Cálculo Diferencial', 'Dra.
Pérez');
materia.materiales.push(new Libro('Cálculo Stewart'));
materia.materiales.push(new Libro('Cálculo Larson'));
lista.push(materia);
return lista;
}
}
class Programacion extends AgregadoDecorator {
obtenerMaterias(): Materia[] {
const lista = super.obtenerMaterias();
const materia = new Materia('PRO101', 'Programación', 'Ing. Torres');
materia.materiales.push(new Libro('Clean Code'));
materia.materiales.push(new Libro('The Pragmatic Programmer'));
materia.materiales.push(new Libro('C# Avanzado'));
lista.push(materia);
return lista;
}
}
class Ingles extends AgregadoDecorator {
obtenerMaterias(): Materia[] {
const lista = super.obtenerMaterias();
const materia = new Materia('ING101', 'Inglés', 'Mtro. García');
materia.materiales.push(new Libro('English Grammar in Use'));
materia.materiales.push(new Libro('Oxford English Dictionary'));
materia.materiales.push(new Libro('Basic English Conversation'));
lista.push(materia);
return lista;
}
}
class Tutoria extends AgregadoDecorator {
obtenerMaterias(): Materia[] {
const lista = super.obtenerMaterias();
const materia = new Materia('TUT101', 'Tutoría', 'Lic. Ramírez');
materia.materiales.push(new Libro('Guía del Estudiante'));
materia.materiales.push(new Libro('Desarrollo Personal y Académico'));
materia.materiales.push(new Libro('Hábitos de Estudio Efectivos'));
lista.push(materia);
return lista;
}
}
```

```
// Interfaz para UI
interface CarreraUI {
id: string;
clave: string;
descripcion: string;
icon: string;
}
const carreras: CarreraUI[] = [
{ id: 'isc', clave: 'ISC', descripcion: 'Ingeniería en Sistemas
Computacionales' },
{ id: 'inf', clave: 'INF', descripcion: 'Ingeniería en Informática' },
{ id: 'icb', clave: 'ICB', descripcion: 'Ingeniería en Ciberseguridad' },
];
export default function App() {
const [carreraSeleccionada, setCarreraSeleccionada] = useState<string>('');
const [materiasSeleccionadas, setMateriasSeleccionadas] = useState<{ [key:
string]: boolean }>({
fisica: false,
calculo: false,
programacion: false,
ingles: false,
tutoria: false,
});
const [planGenerado, setPlanGenerado] = useState<{ carrera: CarreraUI;
clave: string; materias: Materia[] } | null>(null);
const [error, setError] = useState<string>('');
const handleGenerarPlan = () => {
setError('');
if (!carreraSeleccionada) {
setError('Por favor, selecciona una carrera');
return;
}
// Aplicación del PATRÓN DECORATOR
// Crear la carrera base (ConcreteComponent)
let carrera: TecnmComponent;
if (carreraSeleccionada === 'isc') {
carrera = new IngSC();
} else if (carreraSeleccionada === 'inf') {
carrera = new IngI();
} else if (carreraSeleccionada === 'icb') {
carrera = new IngC();
} else {
setError('Selecciona una carrera');
return;
}
// Decorar el componente base con las materias seleccionadas
// Cada decorator envuelve al componente anterior (patrón Decorator)
if (materiasSeleccionadas.fisica) {
carrera = new Fisica(carrera);
}
if (materiasSeleccionadas.calculo) {
carrera = new Calculo(carrera);
}
if (materiasSeleccionadas.programacion) {
carrera = new Programacion(carrera);
}
```

```
if (materiasSeleccionadas.ingles) {
carrera = new Ingles(carrera);
}
if (materiasSeleccionadas.tutoria) {
carrera = new Tutoria(carrera);
}
// Obtener las materias del componente decorado
const materias = carrera.obtenerMaterias();
// Encontrar info UI de la carrera
const carreraUI = carreras.find(c => c.id === carreraSeleccionada);
if (!carreraUI) return;
setPlanGenerado({
carrera: carreraUI,
clave: carrera.clave,
materias: materias
});
// Reset form
setCarreraSeleccionada('');
setMateriasSeleccionadas({
fisica: false,
calculo: false,
programacion: false,
ingles: false,
tutoria: false,
});
};
const toggleMateria = (materia: string) => {
setMateriasSeleccionadas(prev => ({
...prev,
[materia]: !prev[materia],
}));
};
const materiasSeleccionadasCount =
Object.values(materiasSeleccionadas).filter(Boolean).length;
return (
<div className="min-h-screen bg-background relative overflow-hidden">
{/* Decorative background elements */}
<div className="absolute inset-0 overflow-hidden pointer-events-none">
<div className="absolute top-0 right-0 w-96 h-96 bg-secondary/10
rounded-full blur-3xl" />
<div className="absolute bottom-0 left-0 w-96 h-96 bg-accent/10
rounded-full blur-3xl" />
</div>
<div className="relative z-10 max-w-7xl mx-auto px-4 py-12 sm:px-6
lg:px-8">
{/* Header */}
<motion.div
initial={{ opacity: 0, y: -20 }}
animate={{ opacity: 1, y: 0 }}
transition={{ duration: 0.6 }}
className="text-center mb-16"
>
<div className="flex items-center justify-center gap-3 mb-4">
<GraduationCap className="w-12 h-12 text-primary"
strokeWidth={1.5} />
<h1 className="text-5xl md:text-6xl text-primary tracking-tight">
```

```
Sistema de Administración Universitaria
</h1>
</div>
<p className="text-lg text-muted-foreground max-w-2xl mx-auto">
Gestiona tu plan de estudios seleccionando tu carrera y las
materias que deseas cursar
</p>
<div className="mt-6 h-1 w-24 bg-secondary mx-auto rounded-full" />
</motion.div>
{/* Main Form */}
<div className="grid lg:grid-cols-2 gap-8 mb-12">
{/* Selección de Carrera */}
<motion.div
initial={{ opacity: 0, x: -20 }}
animate={{ opacity: 1, x: 0 }}
transition={{ duration: 0.6, delay: 0.2 }}
>
<div className="bg-card rounded-xl shadow-lg border-2
border-border p-8">
<div className="flex items-center gap-3 mb-6">
<div className="w-12 h-12 bg-primary/10 rounded-lg flex
items-center justify-center">
<FileText className="w-6 h-6 text-primary" />
</div>
<h2 className="text-3xl text-primary">Selecciona tu
Carrera</h2>
</div>
<div className="space-y-4">
{carreras.map((carrera, index) => (
<motion.button
key={carrera.id}
initial={{ opacity: 0, x: -10 }}
animate={{ opacity: 1, x: 0 }}
transition={{ delay: 0.3 + index * 0.1 }}
onClick={() => setCarreraSeleccionada(carrera.id)}
className={`w-full text-left p-6 rounded-lg border-2
transition-all duration-300 ${
carreraSeleccionada === carrera.id
? 'bg-primary text-primary-foreground border-primary
shadow-xl scale-105'
: 'bg-card border-border hover:border-primary/50
hover:shadow-md'
}`}
>
<div className="flex items-center gap-4">
<span className="text-4xl">{carrera.icon}</span>
<div className="flex-1">
<div className="font-semibold text-lg
mb-1">{carrera.descripcion}</div>
<div className={`text-sm ${carreraSeleccionada ===
carrera.id ? 'text-primary-foreground/80' : 'text-muted-foreground'}`}>
Clave: {carrera.clave}
</div>
</div>
{carreraSeleccionada === carrera.id && (
<motion.div
```

```
initial={{ scale: 0 }}
animate={{ scale: 1 }}
className="w-6 h-6 bg-secondary rounded-full flex
items-center justify-center"
>
<div className="w-3 h-3 bg-primary-foreground
rounded-full" />
</motion.div>
)}
</div>
</motion.button>
))}
</div>
</div>
</motion.div>
{/* Selección de Materias */}
<motion.div
initial={{ opacity: 0, x: 20 }}
animate={{ opacity: 1, x: 0 }}
transition={{ duration: 0.6, delay: 0.2 }}
>
<div className="bg-card rounded-xl shadow-lg border-2
border-border p-8">
<div className="flex items-center gap-3 mb-6">
<div className="w-12 h-12 bg-accent/10 rounded-lg flex
items-center justify-center">
<Book className="w-6 h-6 text-accent" />
</div>
<div className="flex-1">
<h2 className="text-3xl text-primary">Selecciona
Materias</h2>
{materiasSeleccionadasCount > 0 && (
<p className="text-sm text-muted-foreground mt-1">
{materiasSeleccionadasCount} {materiasSeleccionadasCount
=== 1 ? 'materia seleccionada' : 'materias seleccionadas'}
</p>
)}
</div>
</div>
<div className="space-y-3">
{[
{ key: 'fisica', nombre: 'Física', maestro: 'Dr. López',
clave: 'FIS101' },
{ key: 'calculo', nombre: 'Cálculo Diferencial', maestro:
'Dra. Pérez', clave: 'MAT101' },
{ key: 'programacion', nombre: 'Programación', maestro:
'Ing. Torres', clave: 'PRO101' },
{ key: 'ingles', nombre: 'Inglés', maestro: 'Mtro. García',
clave: 'ING101' },
{ key: 'tutoria', nombre: 'Tutoría', maestro: 'Lic.
Ramírez', clave: 'TUT101' },
].map((data, index) => (
<motion.label
key={data.key}
initial={{ opacity: 0, y: 10 }}
animate={{ opacity: 1, y: 0 }}
```

```
transition={{ delay: 0.4 + index * 0.1 }}
className={`flex items-start gap-4 p-5 rounded-lg border-2
cursor-pointer transition-all duration-300 ${
materiasSeleccionadas[data.key]
? 'bg-accent/5 border-accent shadow-md'
: 'bg-card border-border hover:border-accent/50
hover:bg-accent/5'
}`}
>
<div className="relative flex items-center justify-center
mt-1">
<input
type="checkbox"
checked={materiasSeleccionadas[data.key]}
onChange={() => toggleMateria(data.key)}
className="w-5 h-5 rounded border-2 border-accent
text-accent focus:ring-2 focus:ring-accent/50 transition-all"
/>
</div>
<div className="flex-1">
<div className="font-semibold text-base
mb-1">{data.nombre}</div>
<div className="text-sm text-muted-foreground flex
items-center gap-2">
<User className="w-3.5 h-3.5" />
<span>{data.maestro}</span>
<span className="mx-1">•</span>
<span className="font-mono
text-xs">{data.clave}</span>
</div>
</div>
</motion.label>
))}
</div>
</div>
</motion.div>
</div>
{/* Error Message */}
<AnimatePresence>
{error && (
<motion.div
initial={{ opacity: 0, y: -10 }}
animate={{ opacity: 1, y: 0 }}
exit={{ opacity: 0, y: -10 }}
className="mb-6 p-4 bg-destructive/10 border-2
border-destructive rounded-lg text-destructive text-center font-medium"
>
{error}
</motion.div>
)}
</AnimatePresence>
{/* Generate Button */}
<motion.div
initial={{ opacity: 0, y: 20 }}
animate={{ opacity: 1, y: 0 }}
transition={{ duration: 0.6, delay: 0.4 }}
```

```
className="flex justify-center mb-16"
>
<button
onClick={handleGenerarPlan}
className="group relative px-12 py-5 bg-primary
text-primary-foreground rounded-lg font-semibold text-lg shadow-xl
hover:shadow-2xl transition-all duration-300 hover:scale-105 overflow-hidden"
>
<div className="absolute inset-0 bg-gradient-to-r
from-secondary/20 to-transparent opacity-0 group-hover:opacity-100
transition-opacity duration-300" />
<div className="relative flex items-center gap-3">
<Sparkles className="w-6 h-6" />
<span>Generar Plan de Estudios</span>
</div>
</button>
</motion.div>
{/* Results Section */}
<AnimatePresence mode="wait">
{planGenerado && (
<motion.div
initial={{ opacity: 0, y: 40 }}
animate={{ opacity: 1, y: 0 }}
exit={{ opacity: 0, y: -40 }}
transition={{ duration: 0.6 }}
className="bg-card rounded-xl shadow-2xl border-2
border-primary/20 overflow-hidden"
>
{/* Plan Header */}
<div className="bg-gradient-to-r from-primary to-accent p-8
text-primary-foreground">
<motion.div
initial={{ opacity: 0, x: -20 }}
animate={{ opacity: 1, x: 0 }}
transition={{ delay: 0.2 }}
>
<div className="flex items-center gap-3 mb-2">
<span
className="text-5xl">{planGenerado.carrera.icon}</span>
<div>
<h2 className="text-4xl font-bold
mb-1">{planGenerado.carrera.descripcion}</h2>
<p className="text-xl opacity-90">Clave:
{planGenerado.carrera.clave}</p>
</div>
</div>
</motion.div>
</div>
{/* Materias List */}
<div className="p-8">
<h3 className="text-2xl text-primary mb-6 flex items-center
gap-3">
<div className="h-8 w-1 bg-secondary rounded-full" />
Materias del Plan
</h3>
{planGenerado.materias.length === 0 ? (
```

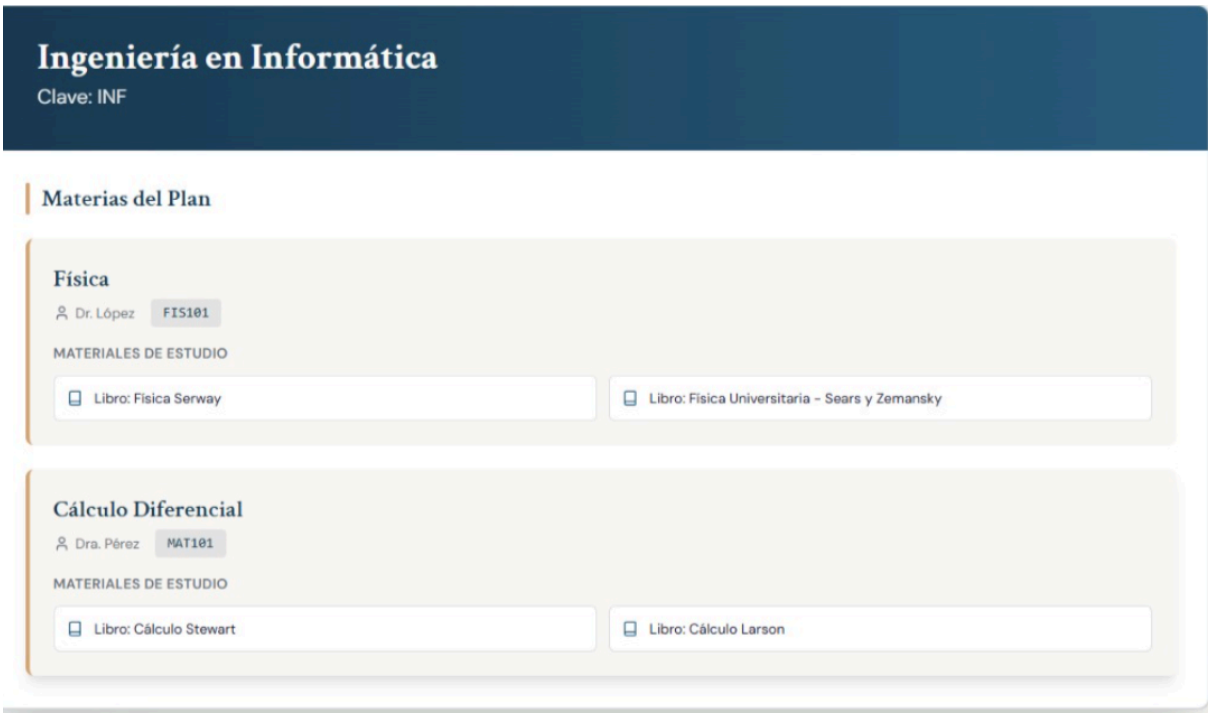
```
<p className="text-muted-foreground text-center py-8">
No se seleccionaron materias para este plan
</p>
) : (
<div className="grid gap-6">
{planGenerado.materias.map((materia, index) => (
<motion.div
key={materia.clave}
initial={{ opacity: 0, x: -20 }}
animate={{ opacity: 1, x: 0 }}
transition={{ delay: 0.3 + index * 0.1 }}
className="bg-muted/50 rounded-lg p-6 border-l-4
border-secondary hover:shadow-lg transition-shadow duration-300"
>
<div className="flex items-start justify-between
mb-4">
<div>
<h4 className="text-2xl text-primary
mb-1">{materia.nombre}</h4>
<div className="flex items-center gap-4 text-sm
text-muted-foreground">
<span className="flex items-center gap-1.5">
<User className="w-4 h-4" />
{materia.maestro}
</span>
<span className="font-mono bg-primary/10
text-primary px-3 py-1 rounded">
{materia.clave}
</span>
</div>
</div>
</div>
{/* Materiales */}
<div>
<h5 className="text-sm font-semibold
text-muted-foreground mb-3 uppercase tracking-wide">
Materiales de Estudio
</h5>
<div className="grid sm:grid-cols-2 gap-3">
{materia.materiales.map((material, idx) => (
<motion.div
key={idx}
initial={{ opacity: 0, scale: 0.9 }}
animate={{ opacity: 1, scale: 1 }}
transition={{ delay: 0.4 + index * 0.1 + idx *
0.05 }}
className="flex items-center gap-3 p-3 bg-card
rounded-md border border-border"
>
<Book className="w-4 h-4 text-accent
flex-shrink-0" />
<span
className="text-sm">{material.obtenerDescripcion()}</span>
</motion.div>
))}
</div>
```

```
</div>
</motion.div>
))}
</div>
)}
</div>
</motion.div>
)}
</AnimatePresence>
{/* Footer */}
<motion.div
initial={{ opacity: 0 }}
animate={{ opacity: 1 }}
transition={{ delay: 1 }}
className="text-center mt-16 text-muted-foreground text-sm"
>
<p>Sistema Universitario TecNM • Gestión Académica</p>
</motion.div>
</div>
</div>
);
}
```

CAPTURAS

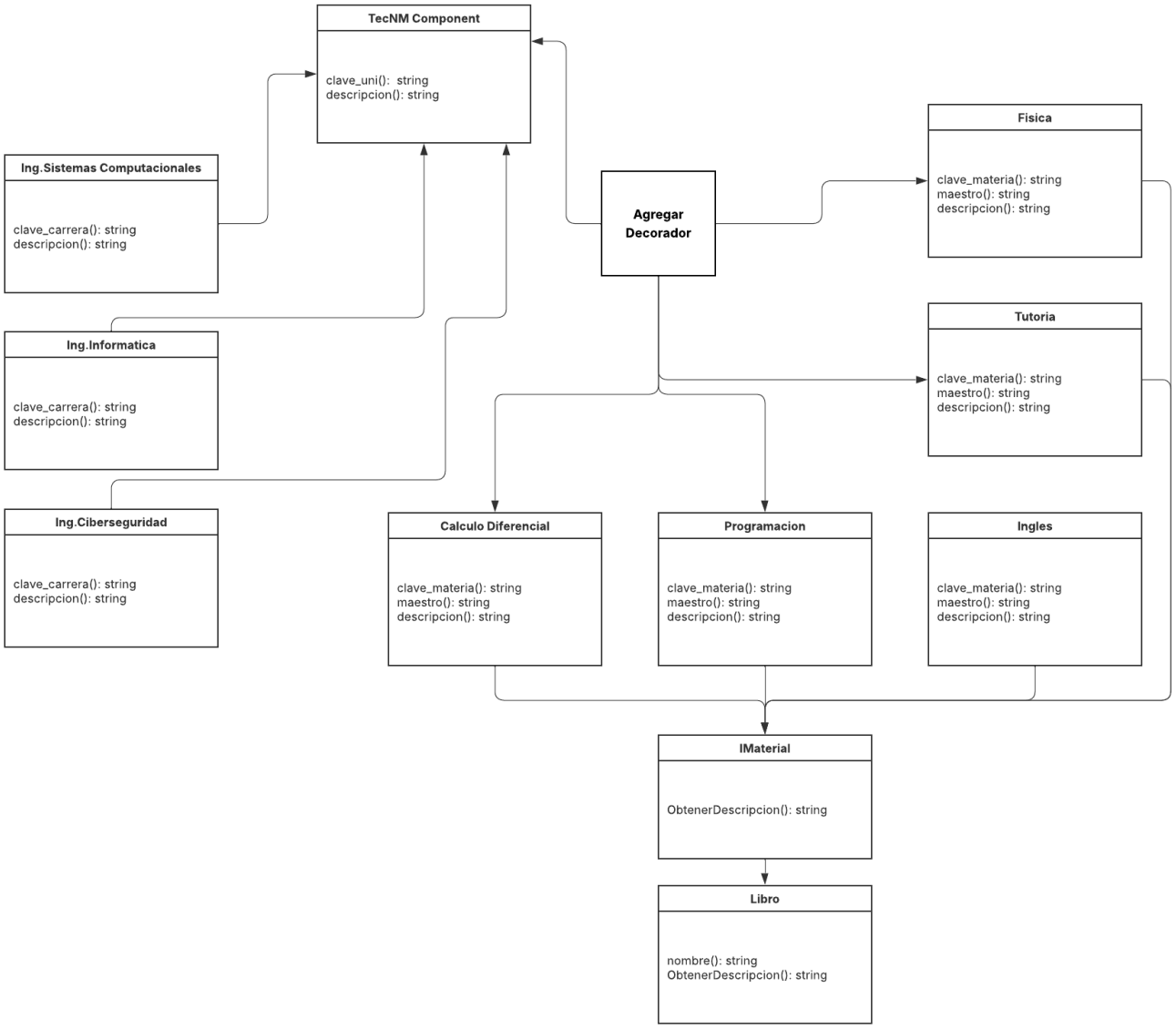
Para crear un plan de estudios es necesario seleccionar la carrera que se va a cursar en la parte de la izquierda y después seleccionar las materias eligiendo las que queramos cursar.

Una vez seleccionadas las opciones de la pantalla anterior se hace click en el botón que dice “Generar Plan de Estudios”, el cual generará la siguiente pantalla



En esta sección podremos ver la carrera y las materias del plan de estudios que creamos previamente, en este se podrán apreciar el nombre de la materia, el maestro, la clave así como los libros necesarios para la clase.

UML


TecNM Component
clave_uni(): string
descripcion(): string
Ing.Sistemas Computacionales
clave_carrera(): string
descripcion(): string
Agregar Decorador
Fisica
clave_materia(): string
maestro(): string
descripcion(): string
Ing.Informatica
clave_carrera(): string
descripcion(): string
Tutoria
clave_materia(): string
maestro(): string
descripcion(): string
Ing.Ciberseguridad
clave_carrera(): string
descripcion(): string
Calculo Diferencial
clave_materia(): string
maestro(): string
descripcion(): string
Programacion
clave_materia(): string
maestro(): string
descripcion(): string
Ingles
clave_materia(): string
maestro(): string
descripcion(): string
IMaterial
ObtenerDescripcion(): string
Libro
nombre(): string
ObtenerDescripcion(): string

Conclusiones

Esmeralda Gamboa Hernandez
-Esta práctica me ayudó a comprender los patrones Decorator y Bridge, en conclusión el decorator funciona para que en el sistema se le agreguen clases pero sin cambiar las clases originales, sería el agregarle las materias, esos serían nuestros decoradores. Por otro lado , el patrón Bridge funciona para que los datos o información de una clase se implementan a nuestro sistema de una manera que no dañe lo anterior realizado, en nuestro caso lo que hicimos fue agregarle a las materias paquetes de libros para que a la hora que el usuario imprima su información le aparezca su carrera, materias y paquetes de libros de estas.

Velazquez Morales Lucero
-En este proyecto pude comprender mejor cómo funcionan los patrones de diseño Decorator y Bridge al aplicarlos al proyecto. El patrón Decorator me permitió agregar materias a una carrera de forma dinámica, sin necesidad de modificar las clases base, lo que hace que el sistema sea más flexible y fácil de entender. Por otro lado, el patrón Bridge me ayudó a separar las materias de sus recursos, en este caso los libros, permitiendo que cada materia tenga sus propios materiales sin depender directamente de ellos. Al combinar ambos patrones me ayudó entender la importancia de diseñar sistemas desacoplados y bien estructurados.

Vidales Hurtado Alitzel Dalhi
-Ambos patrones tienen una estructura bastante similar sin embargo sirven para aspectos diferentes, en la elaboración de este proyecto Decorator ayudó con la base de las materias haciendo más fácil que a futuro se puedan agregar tanto más materias como características de estas mismas y Bridge nos ayudó a asignarle los libros correspondientes a cada una de las materia. Al inicio fue un poco complicado pensar en cómo aplicaríamos correctamente ambos patrones pero consideró que lo logramos de una manera eficiente para el sistema.

Siqueiros Valdes Ivan Antonio
-El Patrón bridge lo veo muy practico y sencillo en su aplicacion asi como su utilidad, ya que simplemente separa el cómo se hace algo del que es para que asi el como se pueda hacer de múltiples maneras, mientras el decorador en la aplicación es más complejo siendo que en su definición suena muy sencillo, el último permitiendo que se puedan extender las clases en vez de modificarlas.